TIG Weld Assessment

Ritwij Parmar | Code (includes weights in the model folders)

Model choice

The weld images contain local texture changes as well as differences in pool shape and boundaries. I started with MobileNetV5 because its convolution, attention and multiscale features seemed suited to capturing both. Its design for efficient visual processing also made it worth testing for future deployment.

I then tried DINOv3 because a strong pretrained representation could be useful with only 12 training recordings. Its image-level and patch-level features let me test local appearance and wider context through a frozen encoder. I wanted to check how well those general features transferred across weld recordings.

With I-JEPA I wanted to learn features from the weld images themselves before classification. Its prediction of hidden-region embeddings from visible context offered a way to learn relationships across the image without using defect labels during adaptation. I compared weld-adapted features with the original pretrained features.

Training approach

During JEPA adaptation the context encoder and predictor learned through gradients. The target encoder followed the context encoder through EMA. I then froze the adapted target encoder and trained a linear classifier separately. The original pretrained encoder was evaluated with the same classifier recipe and recording splits.

Model results

Identical splits: 12 training recordings / 10,338 frames, 6 validation / 6,454 and 9 assessment / 6,763. Complete recordings stay in one split.

Model	Validation (%)			Assessment (%)		
	BA	Recall	FAR	BA	Recall	FAR
MobileNetV5-300M	92.55	88.53	3.43	79.57	81.51	22.38
DINOv3 ViT-L/16	89.70	85.91	6.51	88.00	83.45	7.45
I-JEPA ViT-H/14 - pretrained	94.78	89.72	0.16	84.89	85.02	15.25
I-JEPA ViT-H/14 - adapted	**98.41**	**97.58**	**0.77**	**92.75**	**96.51**	**11.00**

BA: balanced accuracy. Recall: defect recall. FAR: false-alarm rate. All are recording-weighted at threshold 0.5. Assessment was excluded from fitting but inspected in earlier research, so these are retrospective held-out results rather than a new blind test.

Findings

Adapted I-JEPA achieved the highest balanced accuracy and defect recall in this matched assessment. DINOv3 gave fewer false alarms than MobileNetV5 but had lower defect recall than adapted I-JEPA. Lack of penetration was DINOv3's largest source of missed defect frames. Lowering its threshold traded more detections for more false alarms.

The recording breakdown showed a concentrated pattern behind adapted I-JEPA's 11.00% assessment FAR, compared with 0.77% on validation. Recording 170906-141809 contributed 514 of 521 false alarms. The other eight contributed seven and their available good frames had a recording-weighted FAR of 0.48%. That recording also accounted for most of DINOv3's false alarms. SliceFinder associated I-JEPA's errors with bright-region position and contrast. This gave me a specific appearance pattern to investigate without establishing a physical cause or a label error.

I tested targeted augmentation and extra emphasis on difficult good frames through 15-epoch classifier trials with the encoder frozen. I also screened a nonlinear classifier. The original model remained selected because these alternatives did not lower development false alarms while preserving the required defect recall. This made representative recording coverage a useful direction to examine before further model changes.

GPU inference benchmark - selected adapted I-JEPA

Finally, I benchmarked the selected adapted I-JEPA against PyTorch FP32 on the server GPU. TensorRT FP32 gave a 1.77x model-inference speedup and a 1.39x speedup including decoding and preprocessing. Testing latency and memory use on an actual Jetson remains future work, with latency as the main priority.

NVIDIA RTX PRO 6000 Blackwell Server Edition | Matched FP32, TF32 off | Batch 1 | 30 warmups, 200 timings | Median latency

Timing boundary	PyTorch FP32	TensorRT FP32	Speedup
GPU input to returned model logit	14.44 ms	8.17 ms	**1.77x**
PNG bytes to returned decision	22.24 ms	15.96 ms	**1.39x**

TensorRT matched all 6,454 validation decisions against the fresh PyTorch FP32 reference. PNG timing includes decoding and preprocessing but excludes camera acquisition, disk reads and startup. Historical model scores above remain unchanged.

Where I would focus in a real setup

In a real welding setup I would first collect more independent recordings under conditions similar to that case, including good and defective welds. The concentrated pattern makes a gap in recording coverage my first suspicion. I would test that before making further model changes.

My main focus would remain on the models. I would also look for ways to improve the whole pipeline like I did with TensorRT. In a ROS 2 setup I would test whether tuning Fast DDS or switching to Connext improves image delivery and reduces latency. I did similar work in my lab too and it helped a lot.